PRÁCTICA TCP



GP_TCP

Protocolo TCP

Revisión del protocolo TCP mediante el uso de Wireshark.

Evaluable en PL5

PROTOCOLO TCP

1. Introducción y Objetivos

1.1. Introducción

El protocolo TCP es la espina dorsal de la comunicación fiable en Internet. Comprender TCP es una necesidad fundamental para diagnosticar problemas de red, optimizar el rendimiento de las aplicaciones y diseñar sistemas robustos. ¿Por qué una página web carga lenta? ¿Por qué se corta una videoconferencia? ¿Está el problema en el servidor, en el cliente o en la propia red? Muchas veces, la respuesta se encuentra en el comportamiento de TCP.

En esta práctica, estudiaremos en profundidad el protocolo TCP. Usaremos **Wireshark** para examinar trazas reales y ver en acción los algoritmos que permiten a TCP ofrecer fiabilidad y eficiencia sobre la infraestructura de Internet.

No es necesario entregar nada. Esta práctica se evalúa en la prueba de seguimiento PL5.

1.2. Objetivos de Aprendizaje

Al finalizar esta práctica, el alumno será capaz de:

- **Identificar y describir** el establecimiento y cierre de una conexión TCP (*3-way handshake* y terminación).
- **Interpretar y utilizar** correctamente los números de **secuencia (SN)** y de **asentimiento (AN)**.
- **Analizar el control de flujo** y explicar la aparición de ventanas de recepción de tamaño cero **(*Zero Window*)**.
- **Visualizar y cuantificar** el arranque lento (***Slow Start***) utilizando gráficas de Wireshark.
- **Reconocer la transición** desde **evitación de la congestión** tras una pérdida.
- **Distinguir** entre retransmisión por **vencimiento de temporizador RTO**, y **retransmisión rápida**, explicando su impacto en el rendimiento.
- **Usar las herramientas gráficas de Wireshark** (*TCP Stream Graphs*[1], etc.) para validar análisis realizados a nivel de seguimiento individual.

1.3. Material Necesario

- Ordenador con el software **Wireshark** instalado.
- Ficheros de captura disponibles en Aula Virtual:
    - tcp_open_close_with_options.pcap
    - tcp_flow_control_zwp_demo_v2.pcap

[1] https://www.wireshark.org/docs/wsug_html_chunked/ChStatTCPStreamGraphs.html

- tcp_slow_start.pcap
- tcp_rto_reno_v2.pcap
- tcp_slow_start_loss_ca_v2.pcap

3. Análisis Práctico con Wireshark

Cada parte se asocia a una traza concreta y a un conjunto de preguntas. Tus respuestas deben estar relacionadas con las capturas correspondentes.

Parte 1: Apertura y Cierre de la Conexión

En este apartado de la práctica trabajaremos con la traza tcp_open_close_with_options.pcap.

El objetivo es analizar el establecimiento y cierre de una conexión, prestando especial atención a la negociación de parámetros iniciales u opciones TCP (MSS, *Window Scale*, etc.).

3.1.1. Introducción al Escenario

Esta traza (tcp_open_close_with_options.pcap) es el "Hola, Mundo" de TCP. Muestra una conexión muy breve donde un cliente se conecta a un servidor, se establece la sesión e inmediatamente se cierra. Es perfecta para observar establecimiento de la conexión o *handshake* sin interferencias de grandes transferencias de datos, la negociación de parámetros iniciales, y la terminación ordenada de la conexión.

3.1.2. Procedimiento y Cuestiones

Abra el fichero en Wireshark. Active, si no lo está, la opción de números de secuencia relativos:

Edit -> Preferences -> Protocols -> TCP - Relative sequence numbers

y marque la casilla si no aparece marcada.

1. ¿Cuál es la dirección IP y el número de puerto TCP utilizados por el cliente y por el servidor?
2. Indique qué host inicia la conexión y cómo lo deduce.

A continuación, se va a analizar la fase de establecimiento de la conexión.

Observe el primer segmento de la conexión (enviado por el cliente).

3. Indique su número de secuencia relativo.
4. Indique su número de secuencia absoluto.
5. ¿Qué valor hexadecimal muestra la traza para el flag SYN?
6. Expanda la sección de **Opciones TCP**. ¿Qué valor de MSS propone el cliente? Para saber qué es el MSS, consultad este enlace: https://mapamentalar1.web.uah.es/01-notas/03-nivel-de-transporte/tcp/mss-maximum-segment-size.html

Observe el **segundo segmento (SYN-ACK)** enviado por el servidor.

7. ¿Cuál es su número de secuencia relativo?

8. ¿Cuál es el valor de su campo de reconocimiento (AN)?
9. ¿Cómo determina el servidor ese número de asentimiento (AN) a partir de la información recibida en el primer segmento?
10. ¿Qué valor de MSS propone el servidor?

Seleccione el tercer segmento (enviado por el cliente).

11. ¿Cuál es su número de secuencia relativo?
12. ¿Cuál es el valor de su campo de reconocimiento (AN)?
13. ¿Cómo determina el cliente ese AN a partir de la información recibida en el segundo segmento?

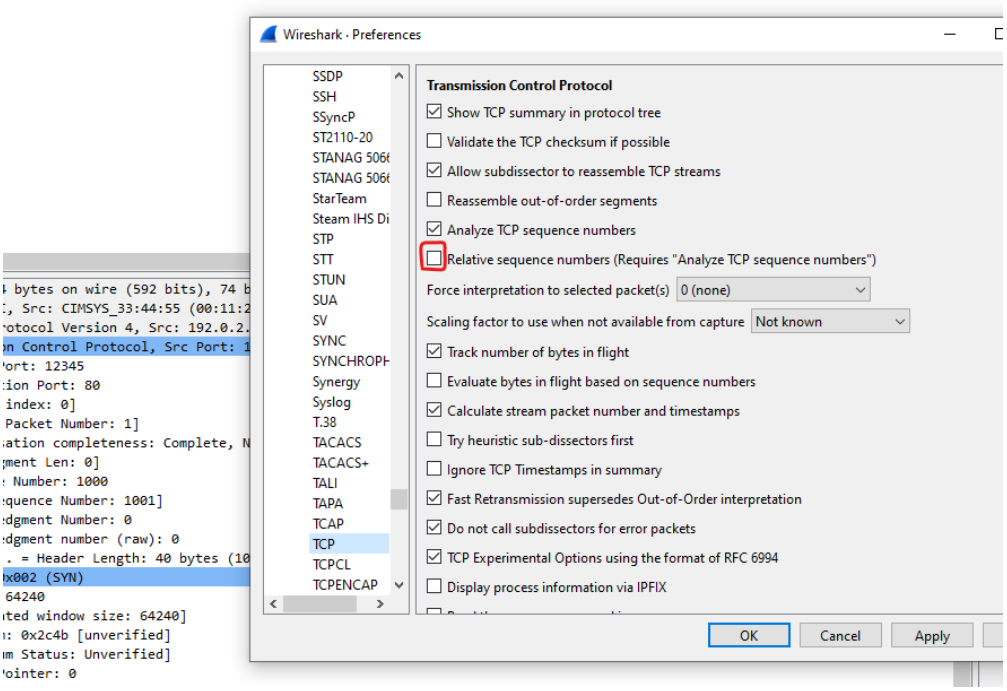
Basándose en la información de los segmentos anteriores:

14. ¿Cuál será el MSS final empleado para la transferencia de datos en esta conexión TCP?

Otra opción importante es *Window Scale*. Esta opción permite superar el límite histórico de 64 KB para el tamaño de la Ventana de Recepción.

15. ¿Está presente esta opción?
16. ¿Qué factor de escala propone el servidor?
17. ¿Qué tamaño de Ventana de Recepción efectivo aceptará el servidor?

Por defecto, Wireshark muestra números de secuencia relativos para facilitar la lectura. Para ver los valores reales, vaya a `Edit` → `Preferences` → `Protocols` → `TCP` y desmarque la casilla *Relative sequence numbers.*



Anote de nuevo el número de secuencia del SYN inicial.

18. ¿A qué se debe la diferencia respecto al valor anterior? (Una vez entendido, puede volver a habilitar los números relativos).

Por último, finalizaremos el análisis de esta traza con el cierre de la conexión. Se recomienda consultar: https://mapamentalar1.web.uah.es/01-notas/03-nivel-de-transporte/tcp/cierre-de-la-conexi%C3%B3n-tcp.html

19. Localice el primer segmento con el flag FIN activo y describa la secuencia completa de segmentos que intervienen en el cierre:

    - Número de pasos observados,
    - Orden en que aparecen FIN y ACK,
    - Quién inicia el cierre (cliente o servidor).

20. Explique por qué, en general, el cierre de una conexión TCP requiere 4 segmentos (secuencia tipo FIN–ACK–FIN–ACK) en lugar de 3 como en el *handshake* de apertura.

Analice el comportamiento del servidor:

21. ¿Por qué no envía su FIN combinado con su primer ACK de respuesta al FIN recibido (lo que produciría un cierre en 3 pasos)?
22. ¿En qué tipo de escenarios sí podría combinar FIN y ACK en un único segmento? Describa al menos un ejemplo razonado.

Parte 2: El Control de Flujo en TCP y la Ventana Cero

En esta parte de la práctica trabajaremos con la captura `tcp_flow_control_zwp_demo_v2.pcap`.

El objetivo es visualizar el control de flujo en una situación donde el receptor se satura y llega a anunciar ventana de recepción cero, y analizar cómo se gestiona esta situación.

3.2.1. Introducción al Escenario

Aquí vemos un cliente enviando datos a un servidor cuya aplicación es "lenta", es decir, no consume los datos del búfer TCP a la misma velocidad a la que llegan. Esto nos permite ver cómo el receptor utiliza el control de flujo para "pedir una pausa" al emisor.

3.2.2. Procedimiento y Cuestiones

Filtre la comunicación para ver solo los paquetes que fluyen del cliente al servidor (el flujo de datos). Use el filtro `ip.src == 192.0.2.1` para ello.

23. ¿Cuál es el tamaño del campo de datos de los primeros 5 segmentos que el cliente envía al servidor?

Ahora, invierta el filtro (`ip.src == 198.51.100.10`) para ver el flujo de ACKs del servidor al cliente. En la columna Info, Wireshark muestra el valor de la ventana de recepción (Win=...).

24. ¿Cuál es el valor de la Ventana de Recepción anunciado por el servidor durante la apertura de la conexión?
25. ¿Cuáles son los valores de la Ventana de Recepción anunciados por el servidor en los siguientes 5 segmentos?

Indique el número de trama que contiene el primer segmento que anuncia **TCP ZeroWindow** (es decir, ventana de recepción de tamaño 0)

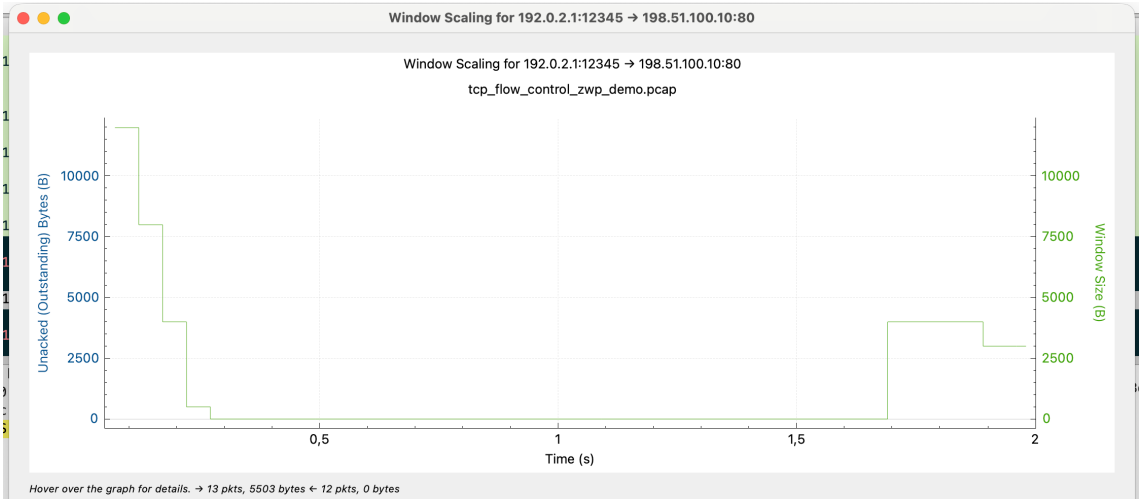
26. ¿Qué significa?

Justo después, el cliente debería dejar de enviar datos. Verifíquelo en la traza.

27. ¿Cómo actúa el cliente ante la recepción del segmento TCP ZeroWindow?
28. ¿Qué tipo de segmentos envía?

El emisor no puede esperar indefinidamente. Busque los segmentos **enviados por el emisor en respuesta a los anteriores (se conocen como ZeroWindowsProbe)**

29. ¿Cada cuánto tiempo los envía el cliente? Indique su longitud y número de secuencia.
30. ¿Para qué envía el cliente estos segmentos? ¿Cómo responde el servidor?
31. ¿Cuáles son los valores de Ventana de Recepción enviados por el servidor en los dos siguientes segmentos inmediatos a los segmentos **ZeroWindowProbe** ACK?
32. ¿Cuál es el valor mínimo de la ventana de recepción anunciada por el receptor en toda la traza? Puede confirmarlo con la gráfica en *Statistics → TCP Stream Graph → Window Scaling.*



(*NOTA: la gráfica que observes en tu equipo puede diferir de esta*)

A continuación, vamos a hacer un análisis del gráfico. Vaya a *Statistics → TCP Stream Graph → Window Scaling*. Haga clic en *"Switch Direction"*. Adjunte una captura de la gráfica y señale en ella claramente las tres fases que ha descrito:

- La disminución de la ventana hasta llegar a cero.
- El periodo de ventana cero, donde el valor se mantiene en 0.
- La recuperación, cuando la ventana vuelve a aumentar.

33. ¿Coincide el periodo de ventana cero de la gráfica con los paquetes **ZeroWindowProbe** que encontró en la traza? Justifique su respuesta.

3.3. Parte 3: Congestión - El Arranque Lento en Acción

En esta parte de la práctica trabajaremos con la traza `tcp_slow_start.pcap`. El objetivo es observar y medir el crecimiento exponencial de la ventana de congestión en la fase de arranque lento.

3.3.1. Introducción al Escenario

Esta captura muestra una transferencia de datos ideal, sin pérdidas y en una red con capacidad suficiente. Es el escenario perfecto para ver el comportamiento del algoritmo de arranque lento, donde TCP intenta descubrir el ancho de banda disponible de la forma más rápida posible.

3.3.2. Procedimiento y Cuestiones

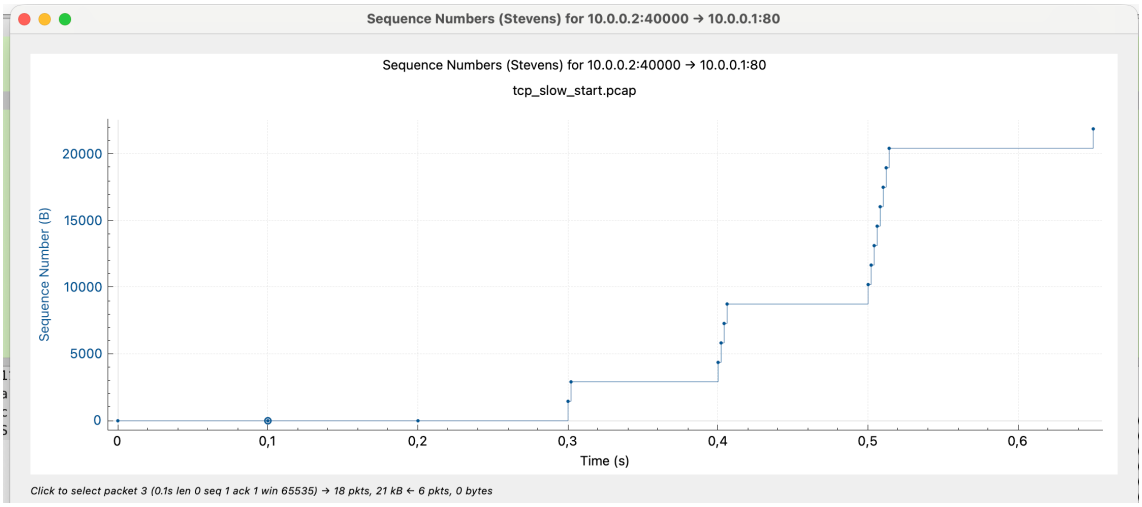
Cuente el número de paquetes enviados desde el cliente (IP 10.0.0.2) al servidor (IP 10.0.0.1) en cada RTT (conjunto de paquetes enviados antes de recibir un nuevo ACK).

34. Rellene la siguiente tabla:

Número de RTT	Nº de Segmentos Enviados
1	
2	
3	
4	

35. ¿Confirma el incremento en el número de segmentos enviados un crecimiento exponencial de la Ventana de Congestión?

A continuación, haremos un análisis gráfico de la evolución de los números de secuencia de los segmentos enviados con respecto a su tiempo de envío, que es la mejor forma de visualizar el rendimiento de TCP. Para obtener esta gráfica vaya a *Statistics* → *TCP Stream Graphs* → *Time-Sequence (Stevens)*. Asegúrese de que está viendo el flujo del servidor al cliente. Haga clic sobre *"Switch Direction"* en caso de que vea un escalón y no el crecimiento exponencial que debe reflejar su respuesta del apartado anterior. Adjunte una captura de la gráfica. La pendiente de esta gráfica representa la velocidad de transmisión.



(*NOTA: la gráfica que observes en tu equipo puede diferir de esta*)

Analice qué forma tiene la curva y responda de manera justificada a las siguientes cuestiones.

36. ¿Qué tipo de crecimiento (lineal o exponencial) representa esa forma?
37. Considerando el primer segmento con datos como el inicio, anote para los siete primeros segmentos de datos: su número de secuencia, el instante de envío y su longitud en bytes.
38. Para cada uno de esos siete segmentos, indique el número de trama del ACK que lo confirma y cuántos bytes de datos reconoce cada uno de esos ACKs.

3.4. Parte 4. La Congestión Golpea: Retransmisión por RTO

En este apartado de la práctica trabajaremos con la traza `tcp_rto_reno_v2.pcap`. El objetivo es diagnosticar una pérdida de paquetes severa que obliga al emisor a esperar a que venza su temporizador de retransmisión (**RTO**).

3.4.1. Introducción al Escenario

Un paquete de datos y su ACK correspondiente se pierden por completo en la red. El emisor se queda esperando una confirmación que nunca llegará. Este es el "peor escenario" de pérdida, y obliga a TCP a tomar la medida más drástica: detenerse, esperar un tiempo prudencial (el RTO) y reiniciar la transmisión de forma muy conservadora.

3.4.2. Procedimiento y Cuestiones

39. Busque en la traza el primer segmento marcado por Wireshark como **TCP Retransmission**. Indique el número de trama que lo transporta, su marca de tiempo y número de secuencia.

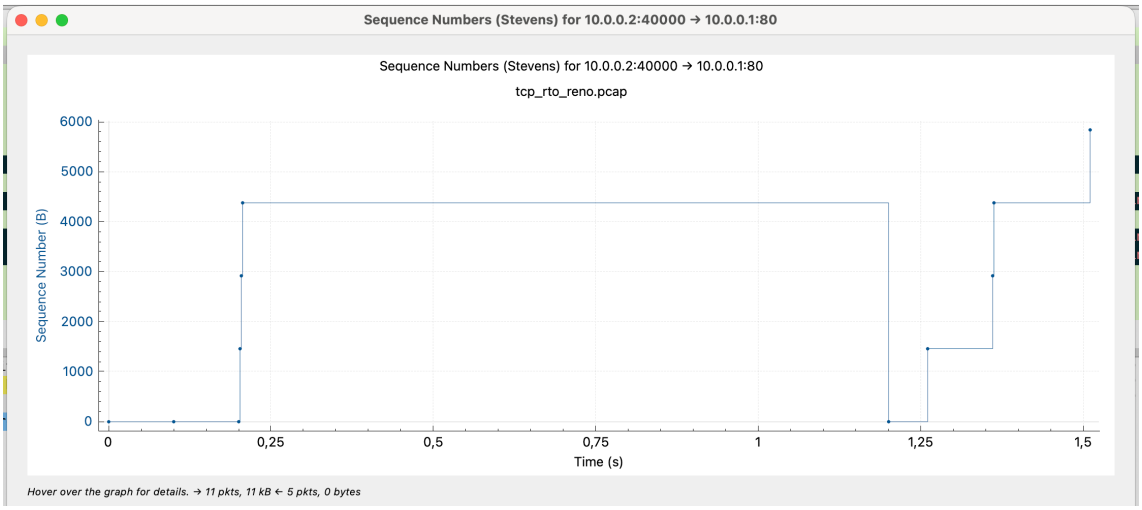
40. Encuentre el segmento **original** con el mismo número de secuencia. Indique el número de trama que lo transporta y su marca de tiempo.

41. Determine el **valor del temporizador de retransmisión RTO**.

42. Observe la actividad del emisor durante el intervalo RTO. ¿Envió algún segmento con ***nuevos*** datos? ¿Por qué esta inactividad es perjudicial para el rendimiento?

43. ¿Qué le ocurre a la Ventana de Congestión justo después de la retransmisión? Para responder, compare el número de segmentos por RTT antes de la pérdida y después. ¿Se ha reiniciado el proceso de arranque lento? Justifique su respuesta.

Genere de nuevo la gráfica para el flujo del servidor al cliente. En la gráfica, debería poder identificar claramente el evento de RTO. Señale en la gráfica los siguientes puntos:



(*NOTA: la gráfica que observes en tu equipo puede diferir de esta*)

- El envío del paquete original que se pierde.
- El largo periodo de inactividad, que se verá como una línea horizontal (el tiempo avanza, pero el número de secuencia no).
- La retransmisión, que aparecerá como un punto en la misma altura (mismo número de secuencia, SEQ) pero en un tiempo posterior.
- El reinicio del arranque lento, visible como una nueva curva con una pendiente inicial muy baja que luego empieza a crecer.

3.5. Parte 5: Pérdida y Recuperación Rápida: De Arranque Lento a Evitación de la Congestión

Para finalizar la práctica trabajaremos con la traza `tcp_slow_start_loss_ca_v2.pcap`

El objetivo es **analizar el mecanismo de recuperación estándar de TCP** (Reno), que es mucho más eficiente que una retransmisión por vencimiento de temporizador RTO, siempre y cuando aquel sea posible.

3.5.1. Introducción al Escenario

Esta es la traza más completa y realista. Vemos una conexión que empieza en arranque lento, sufre una pérdida de paquetes, pero la red es lo suficientemente robusta como para que lleguen paquetes posteriores. Esto permite al receptor informar al emisor de la pérdida de forma implícita, activando una recuperación mucho más rápida y elegante que el RTO.

3.5.2. Procedimiento y Cuestiones

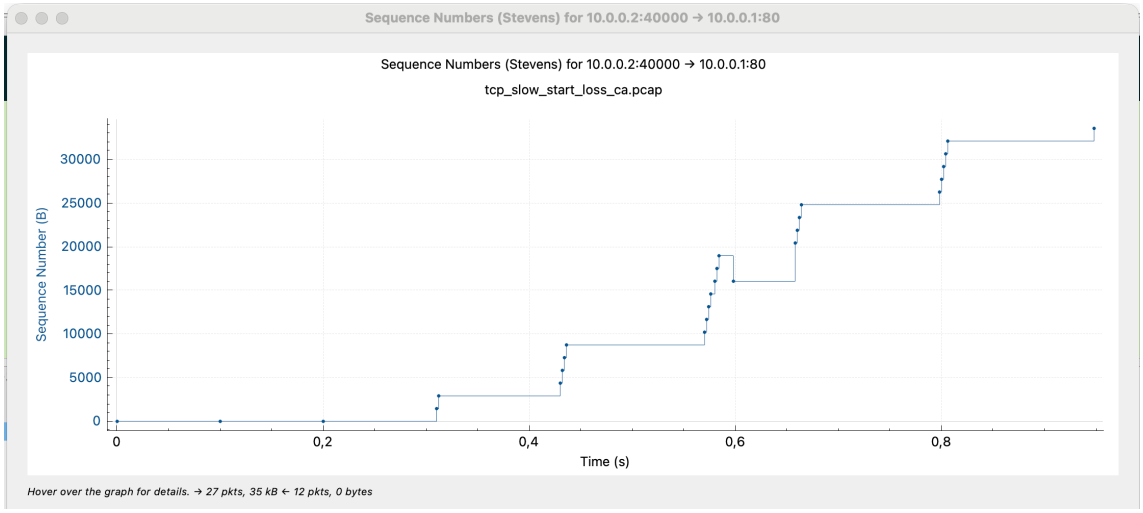
Busque la secuencia de segmentos marcados como **[TCP Dup ACK …]**. Esto es una señal para el emisor de que el receptor ha recibido segmentos posteriores, pero le falta uno intermedio.

44. ¿Qué número de secuencia están asintiendo repetidamente?
45. ¿A qué puede deberse la existencia de ACKs duplicados?

Justo después del tercer ACK duplicado, debería ver un segmento marcado como **[TCP Fast Retransmission]**.

46. Indique el número de secuencia de la trama que lo transporta y el número de secuencia del segmento que se está retransmitiendo.
47. Indique el nuevo valor de la Ventana de Congestión (medida en segmentos) al entrar en Retransmisión Rápida.
48. ¿Se activa este mecanismo antes o después de lo que habría sido un RTO completo (calcule el tiempo y compárelo con el tiempo de la Parte 4)? Justifique su respuesta.
49. Tras esta retransmisión rápida, la conexión entra en **evitación de la congestión**. Observe el envío de segmentos en los dos siguientes RTTs. ¿El crecimiento de la Ventana de Congestión vuelve a ser exponencial como en el arranque lento, o es lineal?

Genere la gráfica **Time-Sequence (Stevens)** para el flujo del servidor al cliente. Esta gráfica es extremadamente reveladora. Adjunte la captura de la gráfica y etiquete las diferentes fases que se indican a continuación sobre la gráfica:



- El crecimiento exponencial inicial, etiquételo como "Fase de Arranque Lento".
- El punto donde se produce la pérdida (la secuencia deja de avanzar momentáneamente y retrocede al número de secuencia del segmento perdido y solicitado mediante Retransmisión Rápida).
- La línea recta con pendiente positiva que sigue a la recuperación, etiquétela como "Fase de Evitación de la Congestión".

50. Explique con sus palabras por qué la gráfica tiene esta forma de "curva seguida de recta". ¿Qué nos dice sobre la inteligencia y adaptabilidad del control de congestión de TCP?

4. Conclusiones

Responda brevemente a las siguientes preguntas para consolidar los conocimientos adquiridos:

1. ¿Cuál es la diferencia fundamental entre el control de flujo y el control de congestión en TCP? ¿Qué campo(s) de la cabecera TCP se asocia(n) a cada uno (directa o indirectamente)?
2. Compare los mecanismos de retransmisión rápida y retransmisión por RTO. ¿Cuál es más eficiente y por qué? ¿En qué situación se ve TCP forzado a usar el RTO?
3. Describa la evolución de la ventana de congestión (VC) a lo largo de una conexión TCP ideal que sufre una única pérdida de paquete gestionada por retransmisión rápida. Mencione las fases por las que pasa (arranque lento, evitación de la congestión) y los eventos que provocan la transición entre ellas.
4. ¿Cuál es el **caudal medio** (bytes por segundo) para el total de la conexión TCP en la traza `tcp_slow_start_loss_ca.pcap`? Explique cómo ha calculado este valor.